

Principles of OOP:

1. Encapsulation - enclosing something in a capsule
 - a. Combine data and methods in a one single unit or class
 - b. Bind / aggregate data together as such to keep it safe
 - c. Examples:
 - i. Private - access to only members of the same class
 - ii. Public - access to all code in the program
 - d. Easier for trouble shooting
2. Inheritance
 - a. Child class inherits the parent class **methods** and **properties**
 - i. Classify all the cats together as having certain common characteristics
 - b. 4 different types of inheritance in csharp
 - c. Reuse code & abstraction of code
 - i. Multiple inheritance is only possible for interface and a base class
3. Abstraction
 - a. In a nutshell, **representing the feature without showing the background details**
 - i. Allow you concentrate on what the object does rather than how it does
 - b. Process of displaying the necessary information
 - i. Only show essential features of an object to the outside world while hiding the other irrelevant information
 - c. Example:
 - i. Make coffee using a coffee machine
 - 1) Does not care how the coffee machine is functioning to prepare me coffee
 - 2) I just need to learn ow to use the machine to make me coffee - all I care about is coffee
 - d. Improve problem solving
4. Polymorphism (many shapes) - one interface multiple functions
 - a. Perform a simple tasks in different ways
 - b. Passing different types and numbers of input values, the output would automatically change
 - c. Classroom --> student, shopping mall --> customers
 - i. --> different behavior
 - d. Example
 - i. Static or compile time polymorphism
 - 1) Method overloading.
 - a) The method/function has the same name but different signatures when overloading. It is also known as compile time polymorphism because the decision to call which method is made at compile time
 - ii. Dynamic or run time polymorphism
 - 1) Dynamic polymorphism is decided at runtime
 - a) Late binding is another term for dynamic/runtime polymorphism
 - b) The method name and signature are used as number of parameters and parameter type must be the same and may have a different implementation
 - c) Dynamic polymorphism is demonstrated by method overriding
 - e. Flexibility to my code

Step1: Interface - blue print - planning abstraction

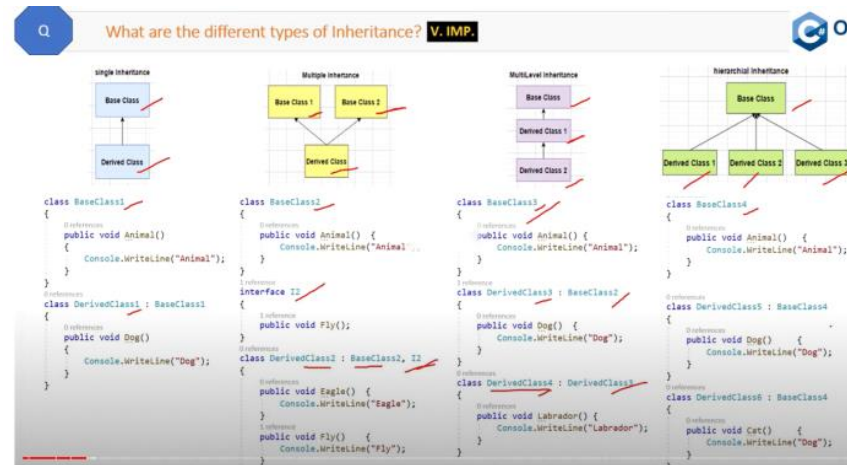
- Empty with shell methods and signature
- Kept with no logic so developers could focus on purely on abstraction, focus on planning, focus on what are the most important things that has to be shown for that entity
- Interfaces are implemented

Step2: abstract class - define common logic

- Abstract class is inherited

Step3: concrete class - full class**Structure of object-oriented programming**

- Classes
 - Blueprints of objects, attributes and methods
- Object
 - An instances of a class
- Methods
 - Describe the behavior of an object
- Attributes
 - Represent the state of an object

Anti-pattern

Screen clipping taken: 07/14/2023 10:42 AM

Interface	Abstract class
It is a contract.	It's a half-defined parent class.
Planning abstraction	Share common logic in child classes
Interface is implemented.	Abstract class is inherited.

Interface could have multiple inheritance whereas abstraction does not.

Interface - everything is abstract, whereas abstract class - some of them are defined whereas some of them are abstract

Example

```

// Abstract class
abstract class Animal
{
    // Abstract method (does not have a body)
    public abstract void animalSound();
    // Regular method
    public void sleep()
    {
        Console.WriteLine("Zzz");
    }
}

// Derived class (inherit from Animal)
class Pig : Animal
{
    public override void animalSound()
    {
        // The body of animalSound() is provided here
        Console.WriteLine("The pig says: wee wee");
    }
}

class Program
{
    static void Main(string[] args)
    {
        Pig myPig = new Pig(); // Create a Pig object
        myPig.animalSound(); // Call the abstract method
        myPig.sleep(); // Call the regular method
    }
}
  
```

Abstraction

Sunday, July 09, 2023 12:09 PM

Abstraction

- a. In a nutshell, representing the feature without showing the background details
 - i. Only show the required things and hide the background details
 - ii. Allow you concentrate on what the object does rather than how it does
- b. Process of displaying the necessary information
 - i. Only show essential features of an object to the outside world while hiding the other irrelevant information
- c. Example:
 - i. Make coffee using a coffee machine
 - 1) Does not care how the coffee machine is functioning to prepare me coffee
 - 2) I just need to learn ow to use the machine to make me coffee - all I care about is coffee
- d. Improve problem solving

Step1: Interface - blue print - planning abstraction

- Empty with shell methods and signature
- Kept with no logic so developers could focus on purely on abstraction, focus on planning, focus on what are the most important things that has to be shown for that entity
- Interfaces are implemented

Step2: abstract class - define common logic - half define parent class having common logic needed for child class

- Abstract class is inherited
 - Some of them at child level and some of them at parent level

Step3: concrete class - full class

Interface	Abstract class
It is a contract.	It's a half-defined parent class.
Planning abstraction	Share common logic in child classes
Interface is implemented.	Abstract class is inherited.

Interface could have multiple inheritance whereas abstraction does not.

Interface - everything is abstract, whereas abstract class - some of them are defined whereas some of them are abstract

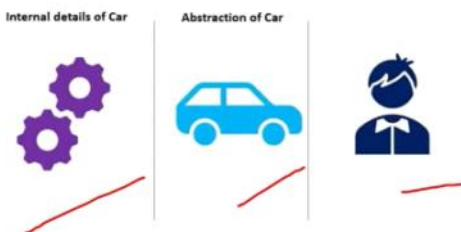
Example

```
// Abstract class
abstract class Animal
{
    // Abstract method (does not have a body)
    public abstract void animalSound();
    // Regular method
    public void sleep()
    {
        Console.WriteLine("Zzz");
    }
}

// Derived class (inherit from Animal)
class Pig : Animal
{
    public override void animalSound()
    {
        // The body of animalSound() is provided here
        Console.WriteLine("The pig says: wee wee");
    }
}

class Program
{
    static void Main(string[] args)
    {
        Pig myPig = new Pig(); // Create a Pig object
        myPig.animalSound(); // Call the abstract method
        myPig.sleep(); // Call the regular method
    }
}
```

- ✦ Abstraction means showing only required things and hide the **BACKGROUND** details.



```
static void Main(string[] args)
{
    Employee employee = new Employee();
    employee.Experience = 5;
    employee.CalculateSalary();
    Console.ReadLine();
}
```

```
public class Employee
{
    2 references
    public int Experience { get; set; }
    1 reference
    public void CalculateSalary()
    {
        int salary = Experience * 300000;
        Console.WriteLine("salary:{0} ", salary);
        CalculateBasicSalary();
        CalculateHra();
    }
    1 reference
    public void CalculateBasicSalary() {
        //some logic
    }
    1 reference
    public void CalculateHra() {
        //some logic
    }
}
```

Background/Hidden methods. Client not aware about them.

Encapsulation

Sunday, July 09, 2023 12:10 PM

Encapsulation - enclosing something in a capsule

- a. **Wrapping** data and methods/properties into a **single** unit
 - i. Combine data and methods in a one single unit or class
- b. Bind / aggregate data together as such to keep it safe
- c. Examples -
 - i. Private - access to only members of the same class
 - ii. Exposing too much for users so we made those complicated methods as private
 - ii. Public - access to all code in the program
 - iii. Fields can be made read-only (if you only use the get method), or write-only (if you only use the set method)
- d. Easier for trouble shooting

Example:

- Collect customer information when they input age
- Class of customerComponentment
 - Age, name, preference
 - You have a function to validate when the users enter the age of the customers
 - Non-negative

Abstraction vs. Encapsulation

Abstraction

- Modeling

Encapsulation

- Implementation

Polymorphism

Sunday, July 09, 2023 3:56 PM

Polymorphism - Many forms/behaviors

- The ability of a variable, object, or function to take on multiple forms

Type of Polymorphism:

1. Method Overloading - compile time polymorphism
 - a. Create multiple methods of the **same name in the same class**, but **different** signatures (all methods work in different way)
 - b. 3 ways of Method overloading
 - i. Different numbers of parameters
 - ii. Different types of parameters (int, double)
 - iii. Different orders of parameters

What is Method Overloading? In how many ways a method can be overloaded?



Method overloading is a type of polymorphism in which we can create multiple methods of the **same name in the same class**, and all methods work in different ways.

*It's a compile time polymorphism.
Why it is called compile time polymorphism, because .Net compiler (CLR) at the time of compiling or building the C# code to machine language knows that the method with same names are different and will do different things.*

Method overloading can be done in 3 ways:

```
public class MethodOverloading
{
    1 reference
    public int Add(int a, int b)
    {
        return a + b;
    }
    1 reference
    public int Add(int a, int b, int c)
    {
        return a + b + c;
    }
    1 reference
    public double Add(double a, double b, int c)
    {
        return a + b + c;
    }
    1 reference
    public double Add(double a, int c, double b)
    {
        return a + b + c;
    }
}
```

1. Number of parameters are different

2. Type of parameters are different

3. Order of parameters are different

2. Method Overriding

- a. Same method name and signature but in different classes
 - i. Having in a child and parent class methods with the **same** name and signature
- b. Keywords
 - i. Base class - virtual
 - ii. Derived class - override

1. Method overloading is a type polymorphism, in which we can create multiple methods of the same name in the same class, and all methods work in different ways.

1. Method overriding is having methods with the SAME NAME and SIGNATURE but in different classes.

2. Method Overloading will not use any such keywords.

2. Overriding uses **VIRTUAL** keyword for base class method and **OVERRIDE** keyword for derived class method.

3. Method overloading doesn't need inheritance. It is in same class.

3. Method overriding **NEEDS INHERITANCE (Base class/ Derived class)**. It is not possible in same class.

```
2 references
public class BaseClass
{
    3 references
    public virtual void Greetings()
    {
        Console.WriteLine("BaseClass Saying Hello!");
    }
}

2 references
public class SubClass : BaseClass
{
    3 references
    public override void Greetings()
    {
        Console.WriteLine("SubClass Saying Hello!");
    }
}
```

Same method name but one is in base class and another is in derived class

```
0 references
static void Main(string[] args)
{
    SubClass objSubClass = new SubClass();
    objSubClass.Greetings();
    Console.ReadLine();
}
```

This will call subclass Greetings() method because of overriding.

//Output: SubClass Saying Hello!

❖ Method overriding is having methods with the SAME NAME and SIGNATURE but in different classes.

❖ In Method Hiding, you can hide the implementation of the methods of a base class from the derived class using the new keyword.

```
public class BaseClass
{
    2 references
    public virtual void Print()
    {
        Console.WriteLine("BaseClass Print");
    }
}

1 reference
public class DerivedClass : BaseClass
{
    2 references
    public override void Print()
    {
        Console.WriteLine("ChildClass Print");
    }
}
```

```
public class BaseClass
{
    1 reference
    public void Print()
    {
        Console.WriteLine("BaseClass Print");
    }
}

1 reference
public class DerivedClass : BaseClass
{
    0 references
    public new void Print()
    {
        Console.WriteLine("ChildClass Print");
    }
}
```

- The "new" keyword mean redefine the method - which has no relationship between the base class method

Inheritance vs. Polymorphism

- Inheritance
 - child class inherits properties and methods from parent class
- Polymorphism
 - Use those methods to perform different tasks

Method overriding:

- Parent class - virtual
- Child class - override

```
5 namespace CSharp
5 {
7     4 references
    public class Person
3     {
        5 references
        public string FullName { get; set; }
        5 references
        public int Age { get; set; }
1
        3 references
        public virtual string GetDetails() => $"{FullName} of age {Age}";
        0 references
        public string GetDetails(double height) => $"{FullName} of age {Age}";
2
    }
5 }
```

```
1 namespace CSharp
1 {
2     2 references
    public class Student:Person
1     {
        2 references
        public int Grade { get; set; }
        4 references
        public override string GetDetails() => $"{FullName} of age {Age}, Grade = {Grade}"
    }
1 }
```

Method Overloading:

- Same function name with different signature

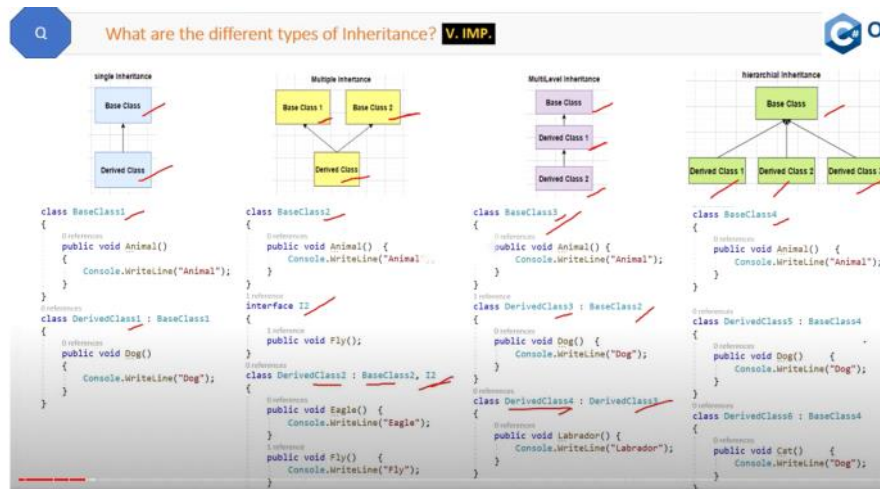
```
1 namespace CSharp
1 {
    4 references
    public class Person
    {
        4 references
        public string FullName { get; set; }
        4 references
        public int Age { get; set; }
        3 references
        public string GetDetails() => $"{FullName} of age {Age}";
        0 references
        public string GetDetails(double height) =>
    }
1 }
```

Inheritance

Friday, July 14, 2023 10:44 AM

Inheritance

- a. Child class inherits the parent class methods and properties
 - i. Classify all the cats together as having certain common characteristics
- b. 4 different types of inheritance in csharp
 - i. Single Inheritance
 - ii. Multiple Inheritance
 - i. Only interface and base class
 - iii. Multiple Level Inheritance
 - iv. Hierarchical Inheritance
- c. Reuse code & abstraction of code
 - i. Multiple inheritance is only possible for interface and a base class
- d. Prevent Inheritance:
 - i. use sealed
 - ii. Use static - static class cannot be inherited from other classes as well
 - i. All members of the class should be static only
 - 1) Mark all properties and methods as static



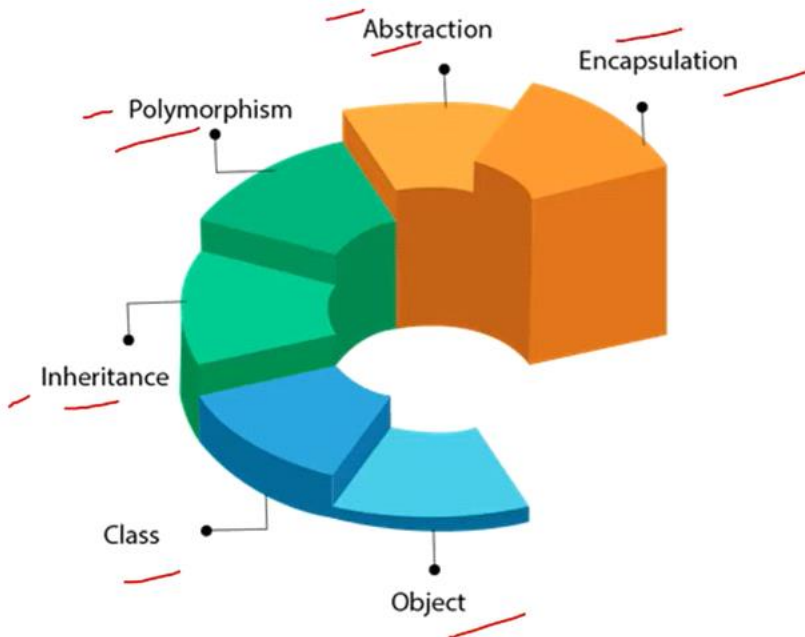
Screen clipping taken: 07/14/2023 10:42 AM

OOPS

Friday, July 14, 2023

9:54 AM

What are the main concepts of OOPS? What are classes and objects?



Class

- A logical unit / blueprint that contains fields, methods, properties

Abstract vs. Interface

Friday, July 14, 2023 12:14 PM

Q

What is the difference between an Abstract class and an Interface (atleast 4)?

V. IMP.

1. Abstract class contains both **DECLARATION & DEFINITION** of methods.

```
public abstract class Employee
{
    0 references
    public Employee()
    {
    }
    private string Name;
    1 reference
    public abstract void Project();
    0 references
    public void Role()
    {
        Console.WriteLine("Software Engineer");
    }
}
```

Method Declared

Method Defined

1. Interface contains **ONLY DECLARATION** of methods.

```
interface IEmployee
{
    1 reference
    public void Project();
    1 reference
    public void Manager();
}
```

Only method Declaration is allowed

2. Abstract class keyword: **ABSTRACT**

2. Interface keyword: **INTERFACE**

3. Abstract class can contain methods, fields, constructor and other class members.

3. Interface can contain **methods only** nothing else.

4. Abstract class does not Support **MULTIPLE INHERITANCE**.

4. Interface supports multiple inheritance.

Similarity

- Abstract class and interface can only use for inheritance not for object creation

When to use Abstract Class

- When you are sure some methods are concrete/defined and must be implemented in the **same** way in all derived classes

When to use interface

- When you know a method has to be there, but it can be implemented **differently** by independent derived classes

Var vs. Dynamic

Friday, July 14, 2023 2:50 PM

Var:

- The type of the variable is decided by the compiler at **compile time**.

Dynamic:

- The type of the variable is decided **at run time**

Q

What is the difference between "var" and "dynamic" in C#?

- ❖ VAR - The type of the variable is decided by the compiler at compile time.

```
0 references
class Program
{
    0 references
    static void Main(string[] args)
    {
        var a = 10;
        a = "Happy";
        Console.WriteLine(a);
        Console.ReadLine();
    }
}
```

- ❖ DYNAMIC - The type of the variable is decided at run time.

```
0 references
class Program
{
    0 references
    static void Main(string[] args)
    {
        dynamic b = 10;
        b = "Happy";
        Console.WriteLine(b);
        Console.ReadLine();
    }
}
```

Collection

Friday, July 14, 2023 3:04 PM

List vs. Array

ARRAY	LINKED LISTS
1. Arrays are stored in contiguous location.	1. Linked lists are not stored in contiguous location.
2. Fixed in size.	2. Dynamic in size.
3. Memory is allocated at compile time.	3. Memory is allocated at run time.
4. Uses less memory than linked lists.	4. Uses more memory because it stores both data and the address of next node.
5. Elements can be accessed easily.	5. Element accessing requires the traversal of whole linked list.
6. Insertion and deletion operation takes time.	6. Insertion and deletion operation is faster.

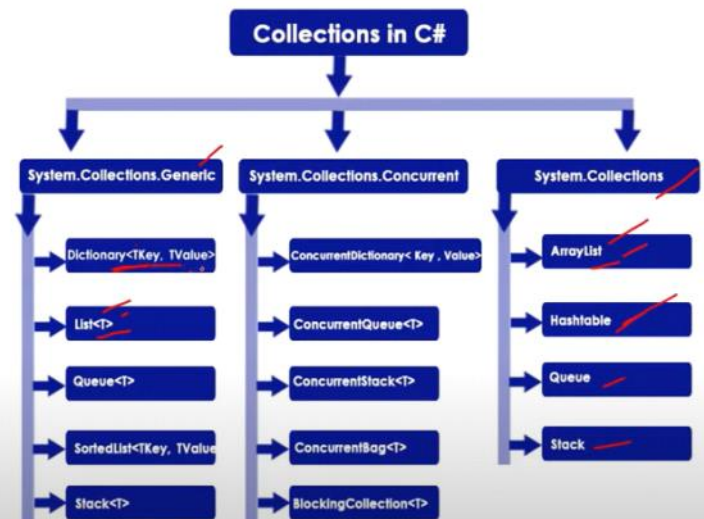
Q

What are Collections in C# and what are their types?



- ❖ C# collection are designed to **store, manage and manipulate** similar DATA more efficiently.
For example ArrayList, Dictionary, List, Hashtable etc.

- ❖ Data manipulation meaning adding, removing, finding, and inserting data in the collection.



❖ Array

1. Array is **STRONGLY** typed. This means that an array can store only specific type of items/ elements.

2. Array can contain **FIXED** number of items.

```

0 references
class Program
{
    0 references
    static void Main(string[] args)
    {
        int[] array;
        array = new int[10];
        array[0] = 1;
        array[1] = "Happy";
    }
}
  
```

❖ ArrayList

1. ArrayList can store **ANY** type of items\elements.

2. ArrayList can store **ANY** number of items.

```

0 references
class Program
{
    0 references
    static void Main(string[] args)
    {
        ArrayList arrayList;
        arrayList = new ArrayList();
        arrayList.Add(1);
        arrayList.Add("Happy");
    }
}
  
```

❖ ArrayList

```

0 references
class Program
{
    0 references
    static void Main(string[] args)
    {
        ArrayList arrList = new ArrayList();
        arrList.Add(7896);
        arrList.Add("Happy");
    }
}
  
```

1. In ArrayList we can only add Items/ Values to the list.

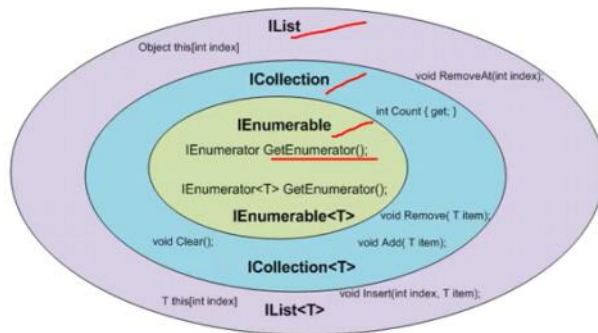
❖ Hashtable

```

0 references
class Program
{
    0 references
    static void Main(string[] args)
    {
        Hashtable hashTable = new Hashtable();
        hashTable.Add("Number", 1);
        hashTable.Add("Car", "Ferrari");
    }
}
  
```

Key Value

1. In Hashtable we can add Items/Values with the Keys.



- ❖ For simplicity, just say that IEnumerable is a box that contains IEnumerator functionality inside it.

```

0 references
class Program
{
    0 references
    static void Main(string[] args)
    {
        var employees = new List<Employee>() {
            new Employee() { Id = 1, Name="Bill" },
            new Employee() { Id = 2, Name="Steve" }
        };

        foreach (var employee in employees)
        {
            Console.WriteLine(employee.Id + ", " + employee.Name);
        }
        Console.ReadLine();
    }
}

```

```

3 references
public class Employee
{
    3 references
    public int Id { get; set; }
    3 references
    public string Name { get; set; }
}

```

SQL

Friday, July 14, 2023 3:04 PM

SQL Basic Commands

1. Data definition language
 - a. Create
 - b. Alter
 - c. Drop
2. Data Manipulation Language
 - a. Select
 - b. Insert
 - c. Update
 - d. Delete
3. Data Control Language
 - a. Grant
 - b. Revoke

SQL Constraints

- Not Null constraints
 - Ensure that a column cannot have a NULL value
- Default constraint
 - Provides a default value for a column when none is specified.
- Unique key
 - Ensures all values in a column are different
- Primary key
 - Uniquely identifies each row/record in a database table
- Foreign Key
 - Uniquely identifies a row/record in any other database table